DESIGN AND VERIFICATION OF A UART CONTROLLED MEMORY INTERFACE

Verilog RTL Project Report

MAIN IMPLEMENTATION

- UART Serial-to-Parallel and Parallel-to-Serial Interface
- Shared 8 × 8 Synchronous FIFO Module (RX and TX Instances)
- Five-State Control FSM / Command Decoder
- 16 × 8 Dual-Port Memory with Circular Addressing

Prepared by: Muhammad Hammad

Project: UART Controlled Memory Interface

Language: Verilog

Date: 08 October 2026

TABLE OF CONTENTS

1. OVERALL SYSTEM ARCHITECTURE 3
2. CONTROL LOGIC FINITE-STATE MACHINE 4
3. MICRO-ARCHITECTURE 5
4. CONCLUSION 6

1. OVERALL SYSTEM ARCHITECTURE

The design implements a UART-controlled 16 × 8 memory interface using UART RX, two instances of a shared 8 × 8 synchronous FIFO module, a five-state control FSM, a 16 × 8 dual-port memory, and UART TX. Baud-rate generation is intentionally omitted; one serial bit is transferred per system-clock cycle. FIFO reads use a registered data output, so command decoding, write-data transfer, and TX loading are sequenced on clock boundaries for deterministic synchronization.

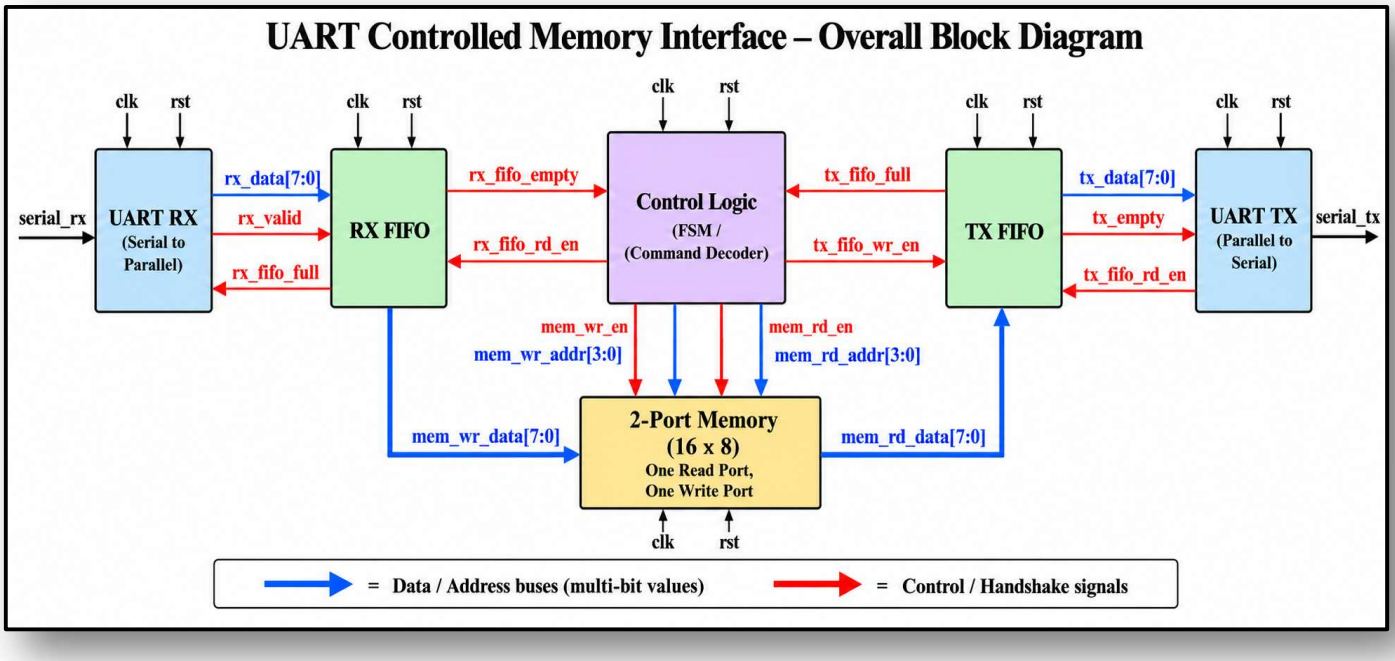


Figure 1. Overall block diagram of the UART controlled memory interface.

UART RX. Receives serial_rx, captures the start bit, eight LSB-first data bits, and stop bit using the IDLE/DATA/STOP FSM, then pulses rx_valid with the completed rx_data[7:0] byte.

RX FIFO. Uses the generic fifo module as an 8 × 8 synchronous buffer. A write stores rx_data when rx_valid is asserted. A read transfers the current FIFO entry into the registered data_out and advances rd_ptr; fifo_empty and fifo_full are generated from the occupancy count.

Control Logic. Implements a five-state, three-block FSM. WAIT_CMD requests the command byte, DECODE_CMD interprets Count[7:5], Address[4:1], and W/R[0], WRITE_FETCH obtains each registered write-data byte, WRITE_DATA performs the synchronous memory write, and READ_DATA transfers memory data into the TX FIFO. The controller maintains current_addr and bytes_left and generates all FIFO and memory control signals.

2-Port Memory. Contains sixteen 8-bit locations. Writes are synchronous to clk through mem_wr_en, mem_wr_addr[3:0], and mem_wr_data[7:0]. Reads are combinational when mem_rd_en is asserted, and mem_rd_data[7:0] feeds the TX FIFO directly. Four-bit address arithmetic naturally wraps from 15 back to 0.

TX FIFO and UART TX. The second instance of the generic FIFO buffers memory-read bytes. UART TX asserts tx_fifo_rd_en while idle, then captures the FIFO's registered tx_data during START before transmitting the eight data bits and STOP bit. This prevents the FIFO pointer from advancing before the transmitter has a valid byte.

2. CONTROL LOGIC FINITE-STATE MACHINE

The updated controller uses five states: WAIT_CMD, DECODE_CMD, WRITE_FETCH, WRITE_DATA, and READ_DATA. The two added states are required by the registered-output FIFO. A FIFO read is requested first; the returned byte is then decoded or consumed on the following state, avoiding same-edge read/use timing conflicts.

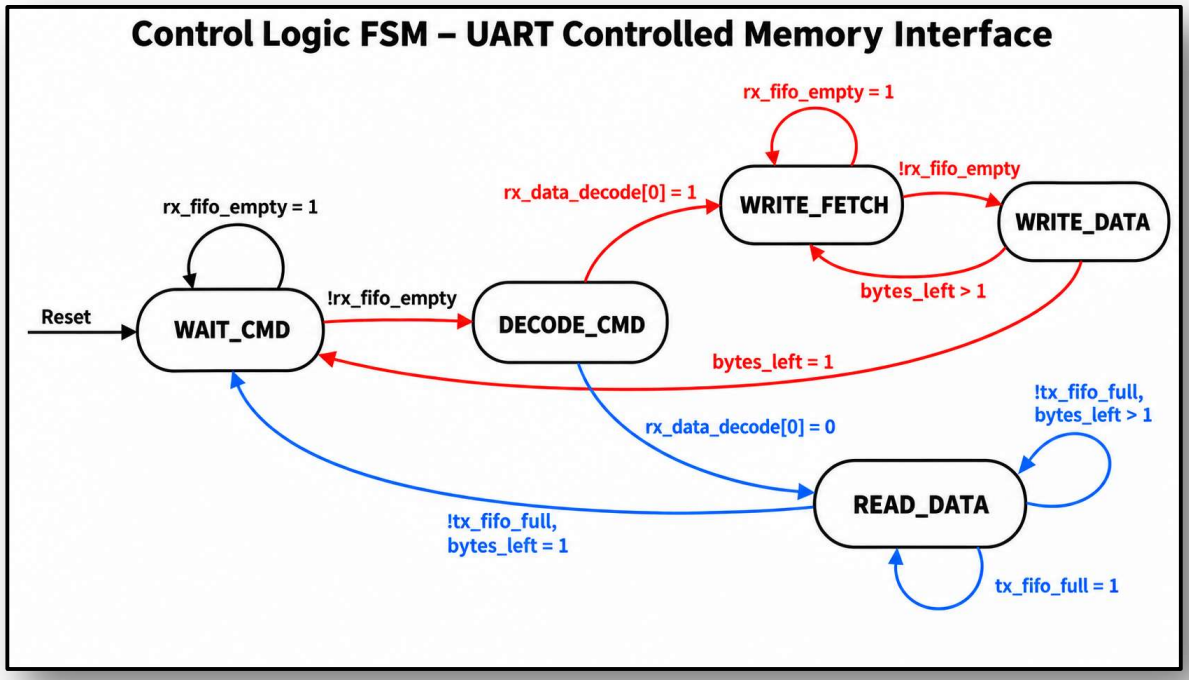


Figure 2. Updated five-state control FSM used with the synchronous registered-output FIFO.

WAIT_CMD. Reset and idle state. If rx_fifo_empty = 1, the FSM remains here. When a command is available, rx_fifo_rd_en is asserted and the next state becomes DECODE_CMD.

DECODE_CMD. The registered FIFO output now contains the command byte. The controller loads current_addr from bits [4:1], sets bytes_left to Count[7:5] + 1, and uses W/R[0] to select WRITE_FETCH for a write command or READ_DATA for a read command.

WRITE_FETCH. Waits for a payload byte in the RX FIFO. When data is available, rx_fifo_rd_en is asserted so the byte is transferred to the FIFO's registered output, then the FSM advances to WRITE_DATA.

WRITE_DATA. Asserts mem_wr_en and writes the fetched byte to mem_wr_addr = current_addr on the rising clock edge. If bytes_left = 1, the transfer is complete and the FSM returns to WAIT_CMD; otherwise the count is decremented, the 4-bit address is incremented, and the FSM returns to WRITE_FETCH.

READ_DATA. If tx_fifo_full = 1, the FSM waits. When space is available, mem_rd_en and tx_fifo_wr_en are asserted together so the combinational memory value is captured by the TX FIFO on the rising edge. The count/address are then advanced or, after the final byte, the FSM returns to WAIT_CMD. Because current_addr is 4 bits, burst accesses wrap automatically from 15 to 0.

3. MICRO-ARCHITECTURE

The updated micro-architecture reflects the actual Verilog modules: a three-state UART receiver, one reusable synchronous FIFO instantiated twice, a five-state three-block controller, a 16 × 8 memory with synchronous write and combinational read, and a four-state UART transmitter. The diagram also shows where the registered FIFO outputs introduce the clock-to-clock sequencing used by the controller and transmitter.

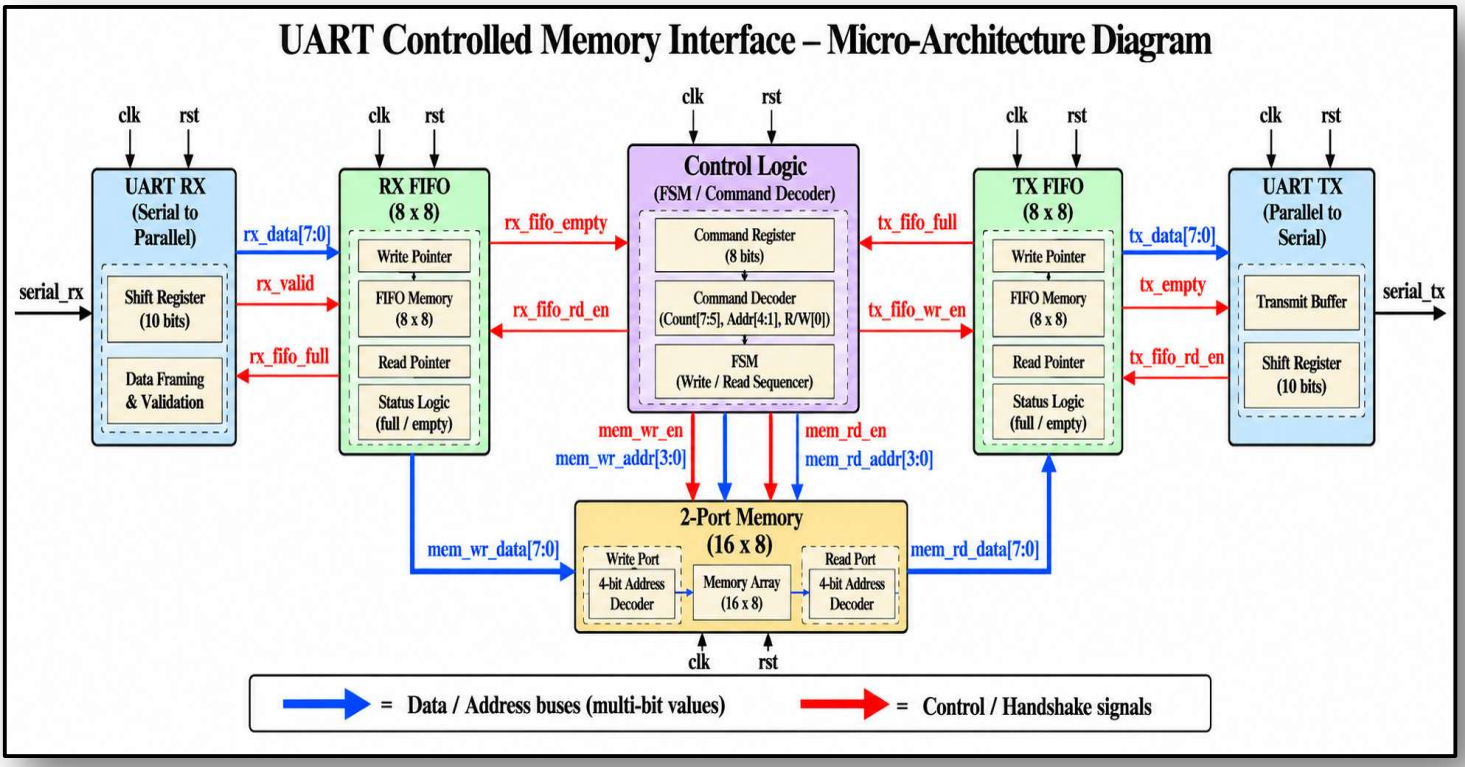


Figure 3. Updated micro-architecture corresponding to the current Verilog modules.

UART RX internals. The receiver uses IDLE, DATA, and STOP states with an 8-bit shift_reg and 3-bit bit_count. A valid stop bit transfers shift_reg to rx_data and pulses rx_valid for one clock, provided the RX FIFO is not full.

FIFO internals. RX and TX use the same fifo module. Each instance contains an 8 × 8 fifo_mem array, 3-bit write/read pointers, a 4-bit count, full/empty flags, synchronous write logic, and a registered data_out. fifo_rd_en therefore produces the requested byte after the active clock edge while also advancing rd_ptr.

Controller internals. The controller is written as three blocks: state/register logic, output logic, and next-state logic. The 3-bit state register selects WAIT_CMD, DECODE_CMD, WRITE_FETCH, WRITE_DATA, or READ_DATA. current_addr/next_addr and bytes_left/next_bytes_left sequence multi-byte commands.

Memory internals. The 16 × 8 array has a synchronous write path and a combinational read path. mem_wr_en stores mem_wr_data at mem_wr_addr on clk, while mem_rd_en exposes memory[mem_rd_addr] as mem_rd_data for capture by the TX FIFO.

UART TX internals. The transmitter uses IDLE, START, DATA, and STOP states with a three-block FSM. In IDLE it requests the next TX FIFO byte; during START it captures the now-valid registered tx_data into shift_reg, DATA serializes bits 0 through 7 using bit_count, and STOP returns the line high before the next byte.

At top level, the same fifo module is instantiated as u_rx_fifo and u_tx_fifo. All internal modules use the same clk, so no additional clock-domain synchronizers are required inside the design; synchronization is achieved by the state sequencing around the registered FIFO outputs and the clocked memory/FIFO writes.

4. CONCLUSION

The updated project implements a modular UART-controlled memory interface in Verilog using a simplified one-bit-per-clock serial receive/transmit scheme, two instances of a shared 8 × 8 synchronous FIFO, a five-state control FSM, and a 16 × 8 dual-port memory. The registered FIFO outputs are handled explicitly by DECODE_CMD and WRITE_FETCH on the receive path and by the START state on the transmit path, ensuring that data is captured only after a FIFO read has completed. The design retains multi-byte read/write commands, automatic 4-bit address wrap-around, FIFO-based flow control, and a clear separation between the payload datapath and control logic.